Experiment 10

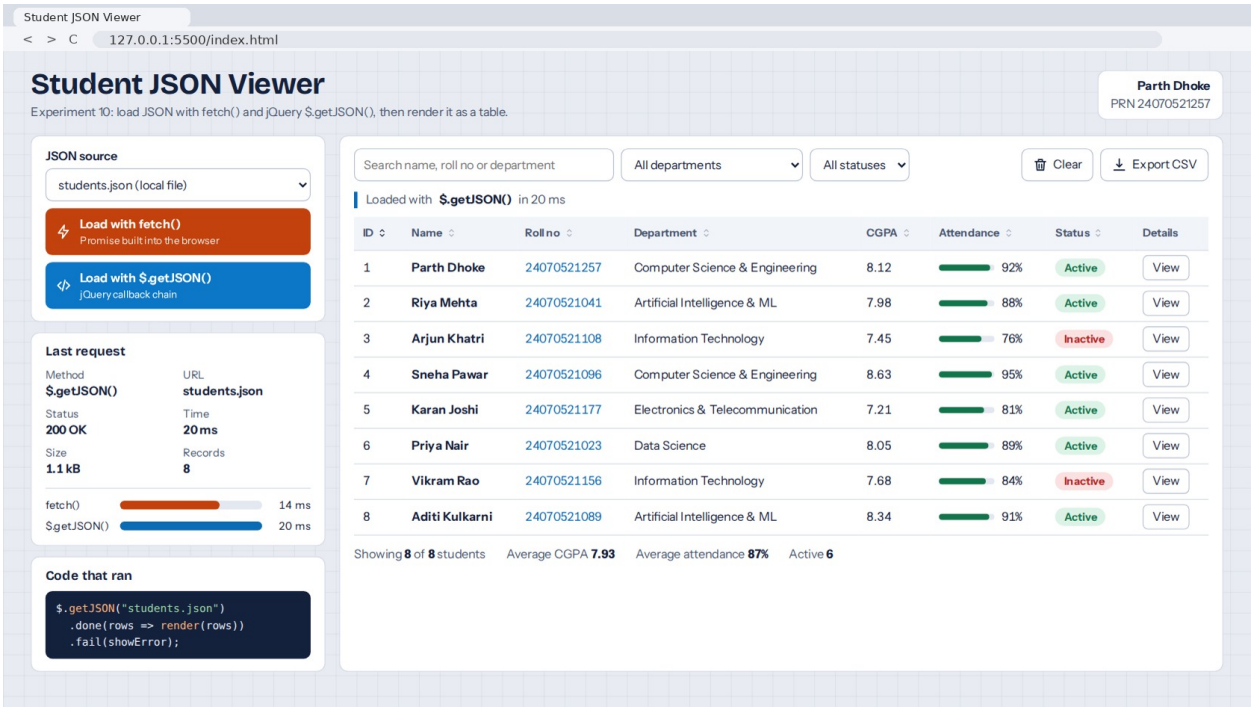
Aim
Load and display JSON data using fetch() and jQuery $.getJSON(); render results dynamically in a table.

Software / Tools Required
- Visual Studio Code (VS Code)
- Google Chrome
- HTML
- CSS
- JavaScript
- jQuery

Theory
Loading and displaying JSON data using JavaScript fetch() and jQuery $.getJSON() is a common technique for retrieving data from a JSON file or web server and presenting it dynamically on a web page.

JSON Format: JSON (JavaScript Object Notation) is a lightweight data format commonly used to store information in the form of objects and arrays. For example, a JSON file may contain student details such as ID, name, course, and marks.
Fetching Data: When the web page needs this information, the `fetch()` method is used to send a request to the JSON file. The `fetch()` method works asynchronously, meaning the browser can request the data without stopping the execution of the rest of the webpage.
Parsing the Response: Once the data is received, the response is converted into a JavaScript object using `response.json()`.
Dynamic Row Generation: The resulting data is processed using methods such as `forEach()`. For each object in the JSON array, JavaScript creates an HTML `<tr>` element containing `<td>` elements for the individual values, and inserts the generated row into the table's `<tbody>` using `innerHTML` or DOM methods.
Scalability: This is called dynamic rendering because the table content is generated automatically from the data rather than being manually written in HTML. If the JSON file contains 10 or 100 student records, the same JavaScript code generates the corresponding number of rows automatically.

Output



The developed webpage provides a Student Information interface where JSON records can be loaded using either fetch() or $.getJSON(), and the results are displayed dynamically in a searchable, filterable, and sortable table.

Result / Conclusion

In conclusion, this experiment demonstrated the loading and display of JSON data using JavaScript fetch() and jQuery $.getJSON(). The application successfully retrieved student records asynchronously, rendered them dynamically in a table, and provided search, filter, and sort functionality along with response latency feedback. Thus, the experiment demonstrated effective use of asynchronous data fetching and dynamic DOM manipulation in a web application.